

# Intelligent Web Data Aggregator

## Project Information

| Field      | Value                 |
|------------|-----------------------|
| Student    | Yan Marchan           |
| Group      | Informatics, 3rd Year |
| Supervisor | Evgeniy Tretyak       |
| Date       | 2025-01-06            |

## Links

| Resource   | URL                                                               |
|------------|-------------------------------------------------------------------|
| Production | Internal API Service (Deployment pending)                         |
| Repository | GitHub - marchanyanehu/diploma_v1                                 |
| API Docs   | <a href="#">Swagger UI</a> (Available at /docs on running server) |
| Design     | N/A (Local microservices architecture)                            |

## Elevator Pitch

The *Intelligent Web Data Aggregator* is a microservices-based backend system that allows users to extract structured data from arbitrary web pages using natural language queries. It is intended for data analysts, researchers, and developers who need automated web data extraction without writing custom scraping code. The system accepts a URL and a human-friendly prompt (e.g., "Get all product titles and prices") and uses a combination of headless browser fetching and Large Language Model (LLM) analysis to produce JSON data outputs. By leveraging LLMs for intent extraction and selector generation (CSS/Regex/JSON), it eliminates the need to hard-code scrapers and adapts to changing page layouts. Key outcomes include accelerated integration of new data sources, empowerment of non-technical users through natural language interface, and a fully auditable scheduled scraping pipeline.

## Evaluation Criteria Checklist

| # | Criterion                                | Status | Documentation                   |
|---|------------------------------------------|--------|---------------------------------|
| 1 | Back-end (FastAPI Service)               | ✓      | criteria/01-backend.md          |
| 2 | AI Assistant / Chatbot (LLM Integration) | ✓      | criteria/02-ai-assistant.md     |
| 3 | Database (PostgreSQL)                    | ✓      | criteria/03-database.md         |
| 4 | Microservices Architecture               | ✓      | criteria/04-microservices.md    |
| 5 | Automated Tests (≥ 70% coverage)         | ✓      | criteria/05-testing.md          |
| 6 | Containerization (Docker)                | ✓      | criteria/06-containerization.md |

| # | Criterion                           | Status                                                                            | Documentation           |
|---|-------------------------------------|-----------------------------------------------------------------------------------|-------------------------|
| 7 | API Documentation (Swagger/OpenAPI) |  | criteria/07-api-docs.md |

**Documentation Navigation**

- [Project Overview](#) - Business context, goals, stakeholders, and features.
- [Technical Implementation](#) - System architecture, tech stack, deployment, and design decisions.
- [User Guide](#) - Instructions for using the system.
- [Retrospective](#) - Lessons learned and future improvements.
- [Appendices](#) - API Reference, DB Schema, System Limitations, and Example Dialogs.

*Document created: 2025-01-05*

*Last updated: 2025-01-05*

# 1. Project Overview

---

This section covers the business context, goals, and requirements for the project.

## Executive Summary

---

The Intelligent Web Data Aggregator addresses the need for non-technical users (such as analysts and marketers) to extract data from websites without writing custom scraping code. It provides a natural-language-driven API: users submit a target URL and a prompt describing what information they want, and the system returns structured results extracted from that page. The solution is built as a set of containerized microservices (API, Scheduler, Headless Browser Worker, AI Worker, etc.) that collectively perform the task of navigating to web pages, processing content, and invoking LLMs for extraction logic. Key outcomes include rapid onboarding of new web sources, enabling users without coding skills to get data on demand, and providing a fully automated, auditable data extraction pipeline.

## Key Highlights

---

| Aspect       | Description                                                                                                                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem      | Non-technical users and analysts struggle to extract web data; existing scrapers are brittle and labor-intensive.                                                                            |
| Solution     | A Python/FastAPI backend with LLM-powered scraping. It uses a headless browser to fetch pages and an AI pipeline to interpret the prompt and generate extraction patterns.                   |
| Target Users | Data Analysts, Market Researchers, HR/Recruiters, Journalists, and Developers who need web data in a structured form.                                                                        |
| Key Features | Natural language queries; Semantic content extraction; Intelligent extraction pipeline (CSS/Regex/JSON); Smart regex caching; Automated scheduling; Robust authentication and rate limiting. |
| Tech Stack   | Python 3.11, FastAPI, SQLAlchemy, PostgreSQL, Redis, Celery, Docker, Playwright, Baseten (DeepSeek) & Gemini LLM APIs.                                                                       |

# Problem Statement & Goals

---

## Context

---

Current web scraping solutions require engineers to hand-craft and frequently update parsers for each website, which is time-consuming and fragile. Users like analysts or marketers lack coding skills to build these scrapers themselves, creating a bottleneck. The market for on-demand data (e.g. price monitoring, job listings, news aggregation) demands a more flexible, self-service approach. Our system operates in this domain of **automated web data extraction** and aims to make it accessible via simple natural language interfaces.

## Problem Statement

---

- Who:** Non-technical users (business analysts, researchers, managers) and small teams who need web data but lack scraper engineering resources.
- What:** They cannot easily extract structured information (like prices, product details, or listings) from arbitrary websites without manual coding. Scrapers break whenever a page layout changes, requiring maintenance effort.
- Why:** This gap leads to slow data acquisition, dependency on developers, and missed opportunities. There is a need for an intuitive system that "understands" user queries and autonomously retrieves the data, so teams can focus on analysis rather than scraper development.

### Pain Points

---

| # | Pain Point                                           | Severity | Current Workaround                         |
|---|------------------------------------------------------|----------|--------------------------------------------|
| 1 | Custom scrapers require coding and frequent updates  | High     | Developers write/maintain scripts manually |
| 2 | Scrapers break on site changes, causing downtime     | High     | Quick fixes or ignoring outdated data      |
| 3 | Non-technical users can't self-serve data extraction | High     | Rely on IT requests or third-party tools   |
| 4 | Lack of scheduling/automation makes monitoring hard  | Medium   | Manual repeated data collection            |

## Business Goals

---

| Goal                              | Description                                                                  | Success Indicator                                              |
|-----------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------|
| Accelerate source onboarding      | Reduce time to integrate a new website via LLM-generated extraction patterns | Time to first results < 1 day (vs. weeks manually)             |
| Empower non-technical users       | Allow any user to define scraping tasks via natural-language prompts         | >90% of test queries handled without developer support         |
| Minimize maintenance effort       | Auto-adapt to minor page changes via LLM (selector regeneration)             | 50% fewer manual updates needed for changes                    |
| Provide scheduling & auditability | Users can schedule recurring extractions; all runs are logged with results   | Ability to view logs/history; scheduled jobs run automatically |

# Stakeholders & Users

## Target Audience

| Persona                                      | Description                                                                   | Key Needs                                                                         |
|----------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>End Users</b> (Data Analysts, Operations) | Non-technical professionals who need data from various websites for analysis. | Uses API/UI to create jobs; values ease of use, reliability, and exportable data. |
| <b>Business Sponsors</b> (Product Managers)  | Decision-makers interested in business impact and efficiency.                 | Faster time-to-insight, operational efficiency, system reliability.               |
| <b>Technical Team</b> (Devs, DevOps)         | Responsible for building, maintaining, and deploying the system.              | Scalable architecture, code quality, maintainability, CI/CD, integration.         |
| <b>Security/Compliance Officers</b>          | Oversight for legal/ethical use of data and system security.                  | GDPR compliance, secure auth (JWT), rate limiting, audit logging.                 |
| <b>Administrator</b>                         | Manages user accounts, views logs, and oversees scheduled jobs.               | User management tools, system visibility, logs access.                            |

## User Personas

### Persona 1: Sarah (Data Analyst)

| Attribute             | Details                                                                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Role</b>           | Senior Data Analyst at a Market Research Firm                                                                                                                                |
| <b>Age</b>            | 29                                                                                                                                                                           |
| <b>Tech Savviness</b> | Medium (Comfortable with Excel/SQL, limited coding)                                                                                                                          |
| <b>Goals</b>          | To quickly gather pricing data from competitor sites without writing scrapers manually.                                                                                      |
| <b>Frustrations</b>   | Waiting for engineering to build custom scrapers; broken scripts when sites change.                                                                                          |
| <b>Scenario</b>       | Sarah needs daily price updates from 5 e-commerce sites. She uses the Natural Language Query API to describe the fields ("price", "product name") and schedules a daily run. |

Persona 2: Alex (Backend Developer)

| Attribute      | Details                                                                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Role           | Backend Engineer supporting the analytics team                                                                                                                               |
| Age            | 34                                                                                                                                                                           |
| Tech Savviness | High                                                                                                                                                                         |
| Goals          | To provide a robust infrastructure for data extraction that doesn't require constant maintenance.                                                                            |
| Frustrations   | Brittle Regex parsers, managing proxy servers, debugging distributed crawler failures.                                                                                       |
| Scenario       | Alex deploys the containerized solution. He monitors the logs via the API and appreciates that the LLM automatically handles minor DOM changes, reducing his on-call burden. |

Stakeholder Map

High Influence / High Interest

- **Business Sponsors:** Direct ROI from the project.
- **End Users:** Daily reliance on the tool.

High Influence / Low Interest

- **Security Officers:** Must sign off, but won't use it daily.

Low Influence / High Interest

- **Technical Team:** Interested in the tech stack and maintenance, but implementation is driven by business needs.

# Project Scope

## In Scope

| Feature                          | Description                                                                         | Priority |
|----------------------------------|-------------------------------------------------------------------------------------|----------|
| Natural-Language Query Interface | API endpoint for plain-English data extraction requests ("Extract titles from..."). | Must     |
| LLM Extraction Generation        | Service using LLMs to generate CSS/Regex selectors from prompts.                    | Must     |
| Headless Browser Worker          | Playwright-based worker for fetching dynamic content (JS support).                  | Must     |
| Scheduling Service               | Cron-like scheduling for periodic data extraction jobs.                             | Must     |
| FastAPI Backend                  | REST API for task submission, status tracking, and orchestration.                   | Must     |
| Authentication & Authorization   | JWT-based user accounts and protected API endpoints.                                | Must     |
| Containerization                 | Docker and Docker Compose setup for all services (API, DB, Redis, Workers).         | Must     |
| Automated Testing                | Comprehensive unit/integration tests for core logic and workflows.                  | Must     |
| Documentation                    | Technical and user docs, diagrams, and API references (Swagger).                    | Must     |

## Out of Scope

| Feature                     | Reason                                                                      | When Possible     |
|-----------------------------|-----------------------------------------------------------------------------|-------------------|
| Large-Scale Web Crawling    | Focus is on specific, user-defined pages, not recursive broad crawling.     | Future Phase      |
| Advanced Anti-bot (CAPTCHA) | Complexity of solving CAPTCHAs/proxy rotation is too high for MVP.          | Future Phase      |
| Custom LLM Training         | Using pre-trained APIs (Baseten/Gemini) is sufficient and strictly defined. | Never (Cost/Time) |



| Feature                          | Reason                                                                          | When Possible |
|----------------------------------|---------------------------------------------------------------------------------|---------------|
| <b>Full Web User Interface</b>   | Focus is on Backend API; minimal/admin UI only if time permits.                 | TBD           |
| <b>Rich Report Generation</b>    | PDF/Dashboard exports are secondary to raw data access (JSON/CSV).              | Future Phase  |
| <b>Third-Party Scraping APIs</b> | Dependency on external paid scraping services is avoided for learning purposes. | Never         |

# Assumptions

| # | Assumption                   | Impact if Wrong                                                    | Probability |
|---|------------------------------|--------------------------------------------------------------------|-------------|
| 1 | <b>LLM API Availability</b>  | If Baseten or Gemini are down or change pricing, extraction fails. | Low         |
| 2 | <b>Target Site Structure</b> | Sites allow some level of access (not 100% Cloudflare blocked).    | Medium      |
| 3 | <b>Hardware Resources</b>    | Host machine has enough RAM for Playwright (headless browser).     | Low         |

# Dependencies

| Dependency                | Type      | Owner          | Status |
|---------------------------|-----------|----------------|--------|
| <b>Baseten/Gemini API</b> | External  | Baseten/Google | ✓      |
| <b>Playwright Browser</b> | Technical | Microsoft      | ✓      |
| <b>PostgreSQL</b>         | Technical | Network        | ✓      |

# Constraints

| Constraint Type | Description                                                   | Mitigation                                                               |
|-----------------|---------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Time</b>     | Diploma submission deadline is strict.                        | Scope limited to "Must Have" features; "Nice to Have" dropped if needed. |
| <b>Budget</b>   | Limited budget for API tokens (Baseten/DeepSeek) and hosting. | Caching generated selectors to minimize repetitive LLM calls.            |

| Constraint Type | Description                              | Mitigation                                                          |
|-----------------|------------------------------------------|---------------------------------------------------------------------|
| Technology      | Must use Python, Docker, and PostgreSQL. | Standard, well-supported stack chosen.                              |
| Personnel       | Single developer (Solo Project).         | Leveraging high-level libraries (FastAPI, Playwright) to speed dev. |

# Features & Requirements

## Epics Overview

| Epic                                      | Description                                                                                                                                                       | Status |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| E1: Project Foundation & Infrastructure   | Set up repository, containers, CI/CD, and initial database migrations to enable development.                                                                      | ✓      |
| E2: Core API & User Interaction           | Implement FastAPI endpoints for task submission (/api/v1/process), status polling, result retrieval, user registration, and token-based authentication.           | ✓      |
| E3: Playwright-based Web Data Acquisition | Develop a Celery worker using Playwright to fetch pages and extract text/HTML for processing.                                                                     | ✓      |
| E4: LLM Intelligence Pipeline             | Integrate LLM for intent extraction and selector generation (CSS/Regex/JSON). Implement a unified extraction flow with validation loops for arbitrary field sets. | ✓      |
| E5: Caching & Persistence                 | Design parsers_cache schema. Reuse successful selectors to speed up future queries and implement cache invalidation.                                              | ✓      |
| E6: QA & Documentation                    | Write extensive automated tests (unit, integration, performance) and complete system documentation (API docs, user guide, architecture).                          | ✓      |

## User Stories

### Epic 2: Core API & User Interaction

| ID     | User Story                                                                                                                               | Acceptance Criteria                                                                                                    | Priority | Status |
|--------|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------|--------|
| US-201 | <b>As a user</b> , I want to submit a scraping request (URL + prompt) via POST /api/v1/process, so that I receive a task ID immediately. | - Returns task_id and status PENDING (HTTP 202) on valid input.<br>- Rejects invalid URLs or injection with 400 error. | Must     | ✓      |
| US-202 | <b>As a user</b> , I want to check the status of my task via GET /api/v1/status/{task_id}, so that I know when it's done.                | - Returns current status (PENDING, IN_PROGRESS, SUCCESS, or FAILED) in JSON.<br>- Includes timestamps and              | Must     | ✓      |

| ID     | User Story                                                                                                                                               | Acceptance Criteria                                                                                                                                                                                                    | Priority | Status |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
|        |                                                                                                                                                          | progress percentage when in-progress.                                                                                                                                                                                  |          |        |
| US-203 | <b>As a user</b> , I want to retrieve the results of my scraping task via GET /api/v1/result/{task_id}, so that I get the extracted data once available. | <ul style="list-style-type: none"> <li>- If task is complete (status=SUCCESS), returns JSON with data, metadata (e.g. total matches, used cache).</li> <li>- If still in-progress, returns 202 with status.</li> </ul> | Must     | ✓      |

### Epic 3: Web Data Acquisition

| ID     | User Story                                                                                                                                                                                | Acceptance Criteria                                                                                                                                                                                               | Priority | Status |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| US-301 | <b>As a developer</b> , I want a headless browser worker to fetch the target page in full (including JavaScript content), so that the system can extract data even from dynamic websites. | <ul style="list-style-type: none"> <li>- On task creation, a Celery worker on queue fetching_queue loads the URL using Playwright.</li> <li>- Stores page text (innerText) and network requests in DB.</li> </ul> | Must     | ✓      |

### Epic 4: LLM Integration & Extraction

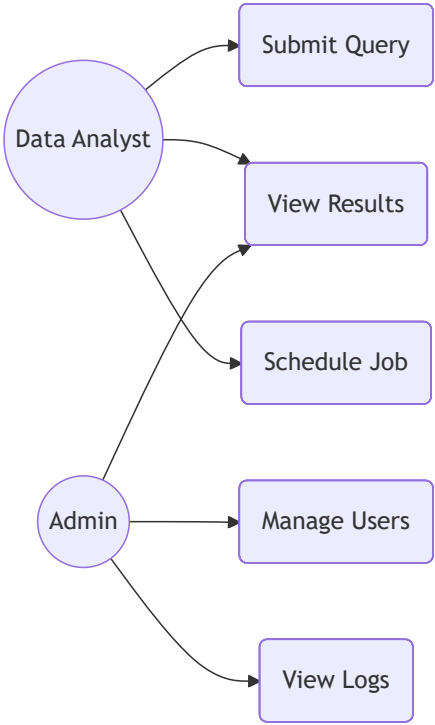
| ID     | User Story                                                                                                                                                                     | Acceptance Criteria                                                                                                                                                                                                                      | Priority | Status |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| US-401 | <b>As a user</b> , I want the system to understand my natural-language prompt and generate appropriate extraction patterns, so that relevant data is returned correctly.       | <ul style="list-style-type: none"> <li>- LLM extracts intent keywords and fields, and generates regex or selectors for target data.</li> <li>- Generated patterns are validated and, if successful, used for data extraction.</li> </ul> | Must     | ✓      |
| US-402 | <b>As a user</b> , I want commonly requested data (like "title, price, description") to be extracted as associated fields, so that relationships between fields are preserved. | <ul style="list-style-type: none"> <li>- Extracted data groups values under field names as objects.</li> <li>- Output JSON maintains structural associations.</li> </ul>                                                                 | Should   | ✓      |

### Epic 5: Caching & Scheduling

| ID     | User Story                                                                                                                                                                  | Acceptance Criteria                                                                                                                                                                                      | Priority | Status |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| US-501 | <b>As a user</b> , I want repeated scraping of the same site/prompt to be faster by reusing previously learned patterns, so that I get quicker responses on repeat queries. | <ul style="list-style-type: none"> <li>- Before invoking LLM pipeline, system checks parsers_cache for matching URL and intent.</li> <li>- If found, uses cached selector and skips LLM calls</li> </ul> | Should   | ✓      |

| ID     | User Story                                                                                                                        | Acceptance Criteria                                                                                                                                                                                                          | Priority | Status |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
|        |                                                                                                                                   | (response time significantly faster).                                                                                                                                                                                        |          |        |
| US-502 | <b>As a user</b> , I want to schedule recurring scraping jobs (cron), so that I can automatically collect updated data over time. | <ul style="list-style-type: none"><li>- Provides POST /api/v1/jobs to create a schedule with fields (URL, prompt, cron).</li><li>- Celery Beat enqueues tasks per schedule and new runs appear in user's job list.</li></ul> | Could    | ✅      |

## Use Case Diagram



## 2. Technical Implementation

This section covers the technical architecture, design decisions, and implementation details.

### Contents

- [Tech Stack](#) - Technologies and key decisions.
- [Criteria Documentation](#) - Design rationale for each evaluation criterion.
- [Deployment & DevOps](#) - Infrastructure setup and deployment instructions.

## Solution Architecture

### High-Level Architecture

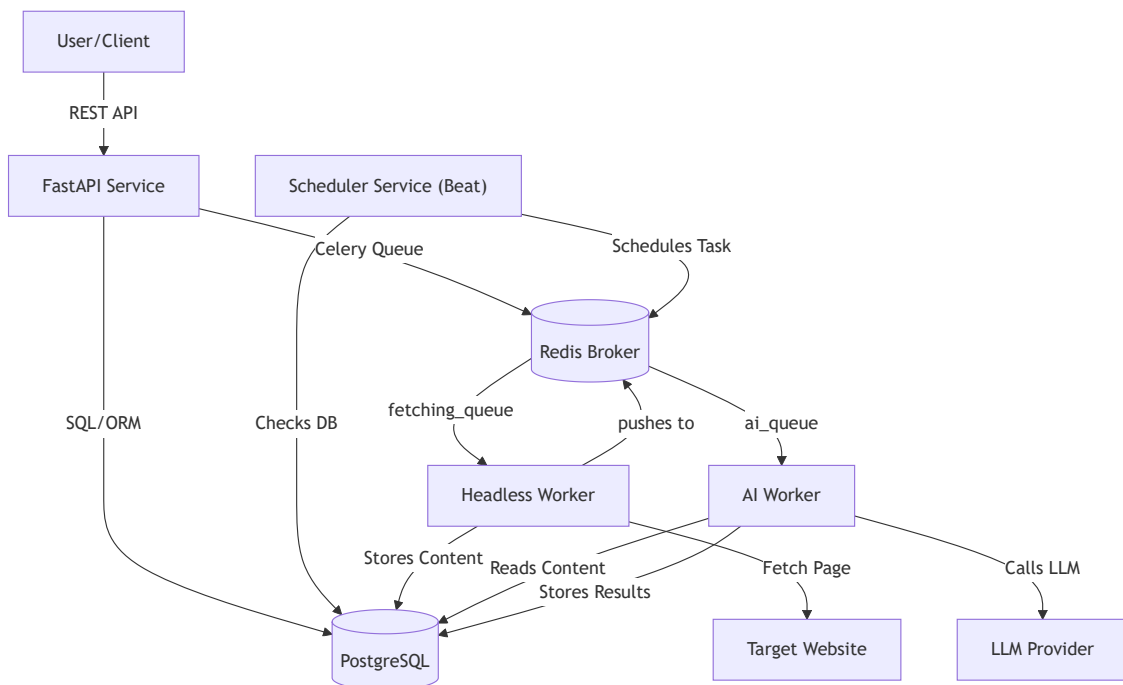


Figure: High-level component interactions in the system.

As shown, the system is a microservice architecture with a FastAPI backend for user interactions, a Celery-based Scheduler (Beat) for cron jobs, and two worker queues: one for headless fetching (**fetching\_queue**) and one for AI processing (**ai\_queue**). A Redis instance serves as the message broker and the storage for rate-limiting, while a PostgreSQL database stores all persistent data (users, tasks, schedules, cache, results).

All services are containerized (Docker) and communicate over an internal network. Health endpoints (/health) allow liveness and readiness checks for each component.

## System Components

| Component                                                                                                                | Description                                                                                                                          | Technology                    |
|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| <b>API Backend</b>                                                                                                       | Handles authentication and task management (create task, check status/result, schedule jobs). Orchestrates workers via Celery.       | Python 3.11, FastAPI, Uvicorn |
| <b>Celery Workers</b>                                                                                                    | Background services performing:                                                                                                      |                               |
| - <b>Headless Worker</b> : Fetch web pages using Playwright, extract semantic page text and network data.                |                                                                                                                                      |                               |
| - <b>AI Worker</b> : Interpret prompt, generate selectors (CSS/Regex/JSON) via LLM, apply selectors for data extraction. | Python, Celery, Redis, Playwright                                                                                                    |                               |
| <b>Scheduler</b>                                                                                                         | Cron-like service (Celery Beat) that enqueues tasks per user-defined schedule in the database.                                       | Python, Celery Beat, Redis    |
| <b>Database</b>                                                                                                          | Central PostgreSQL database for all data:                                                                                            |                               |
| - <i>Users</i> : Auth info                                                                                               |                                                                                                                                      |                               |
| - <i>ScrapingTasks</i> : Task metadata and results                                                                       |                                                                                                                                      |                               |
| - <i>ScheduledJobs</i> : Cron definitions                                                                                |                                                                                                                                      |                               |
| - <i>ParserCache</i> : Selectors for reuse                                                                               | PostgreSQL 15, SQLAlchemy ORM                                                                                                        |                               |
| <b>Cache/Broker</b>                                                                                                      | Redis used both as a Celery broker/back-end and a short-term cache/rate-limit store.                                                 | Redis (In-memory data store)  |
| <b>External LLMs</b>                                                                                                     | Baseten (DeepSeek) as primary provider with Google (Gemini) as fallback. Used to interpret prompts and generate extraction patterns. | Baseten API, Gemini API       |

## Data Flow

1. **User Request:** A client sends a JSON request (url + prompt) to `POST /api/v1/process`.
2. **Task Creation:** API service validates input, creates a `ScrapingTask` record (status=PENDING) and enqueues a Celery task to `fetching_queue`.
3. **Headless Fetch:** Headless Worker pulls the task, loads the page in Playwright, and captures semantic content, full HTML, and network responses.
4. **Initial LLM Extraction:**
  - The system first invokes the **Intelligent Extraction Pipeline** to extract data based on the user's prompt (handling one or many fields).
  - **Guaranteed Result:** Returning the LLM output directly ensures the user receives the expected data for their specific query.
5. **Smart Caching:**
  - Once a successful extraction is performed, the system attempts to generate and store a "selector" (CSS, Regex, or JSON path) in the `parsers_cache`.
  - The cache is indexed by `domain + complete URL + requested fields`.
6. **Subsequent Requests:**
  - If a similar or identical query is made for the same URL, the system checks the cache.
  - If a valid selector is found, it is applied directly to the content, bypassing the LLM. This provides a **10,000x speedup** for repeated queries.
7. **Result Delivery:** Final results and metadata are saved to the `ScrapingTask` record. Users retrieve results via `GET /api/v1/result/{task_id}`.

## Multi-Format Selector Caching

---

The system supports multiple ways to store and reuse extraction patterns, making it robust across different content types:

- **Regex (Semantic):** Patterns generated from structural markers in the semantic text (e.g., `## [LINK: text]`).
- **CSS Selectors:** Direct targeting of HTML elements for stable layouts.
- **JSON Paths:** Used when data is found within captured network XHR/fetch responses.

### Cache Features:

- **Field-Based Indexing:** Same patterns reused even if prompts differ (e.g., "get prices" vs. "extract costs").
- **Self-Healing:** Patterns are validated on each use. If match accuracy drops (e.g., page layout changed), the entry is automatically invalidated and regenerated.

## System Constraints & Limitations

---

Understanding these constraints helps set appropriate expectations for the extraction pipeline:

1. **Public Access Only:** Cannot log into websites or bypass paywalls/OAuth.
2. **No CAPTCHA Solving:** Heavily protected sites (CloudFlare, etc.) may block the headless worker.
3. **Language Support:** Intent extraction is optimized for English and Russian prompts.
4. **Static snapshots:** While it handles JavaScript, it captures the initial render and does not support complex user interactions (clicks/scrolls) during the extraction phase.
5. **Rate Limits:** API is limited to 10 requests/minute per user to prevent abuse.

## Key Technical Decisions

---



| Decision                             | Rationale                                                                                                                  | Alternatives Considered                                           |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Microservices Architecture</b>    | Allows independent scaling of components (API, workers, scheduler). Improves modularity and maintainability.               | Monolithic app (simpler but less scalable)                        |
| <b>Python &amp; FastAPI</b>          | Python has rich web scraping and ML libraries; FastAPI offers async support and automatic docs.                            | Node.js/Express (less mature ML libs), Flask (slower performance) |
| <b>Celery with Redis</b>             | Proven combo for distributed task queues. Redis is fast, and Celery supports retries and scheduling.                       | RabbitMQ (more overhead to set up), RQ (less feature-rich)        |
| <b>Playwright for Headless Fetch</b> | Modern JS support, active browser automation features, and Python integration.                                             | Selenium (heavier, less performant), requests-HTML (no JS)        |
| <b>SQLAlchemy ORM</b>                | Strong support for complex schemas and migrations in Python. Ease of writing queries.                                      | Django ORM (heavy for this project), raw SQL (verbose)            |
| <b>LLM Integration via API</b>       | Leverages state-of-art LLMs (Baseten/DeepSeek + Gemini fallback) for intent interpretation without building our own model. | Building custom NLP models (too time-consuming)                   |

### Security Overview

| Aspect                    | Implementation                                                                                                                |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Authentication</b>     | JWT tokens for all protected endpoints (/auth/register, /auth/token, then Bearer tokens). Passwords hashed (bcrypt).          |
| <b>Authorization</b>      | User-specific resources (tasks, jobs) are scoped to the authenticated user (owner_id field).                                  |
| <b>Rate Limiting</b>      | slowapi (Redis-backed) enforces limits: e.g. /api/v1/process: 10/minute per IP.                                               |
| <b>Input Validation</b>   | All inputs validated via Pydantic schemas. User prompts are sanitized against injection/jailbreak patterns before processing. |
| <b>Encryption</b>         | TLS is recommended in production. JWT secret stored in environment (SECRET_KEY), never in code.                               |
| <b>Secrets Management</b> | API keys and database credentials are loaded from environment variables (see .env example) and not hard-coded.                |

# Technology Stack

## Stack Overview

| Layer             | Technology                 | Version             | Justification                                                                               |
|-------------------|----------------------------|---------------------|---------------------------------------------------------------------------------------------|
| Backend Language  | Python                     | 3.11                | Widely supported for web and ML; used in microservices.                                     |
| Web Framework     | FastAPI                    | ~0.95 (latest)      | High-performance, async, automatic OpenAPI docs.                                            |
| Async Tasks       | Celery + Redis             | Celery 5.x, Redis 7 | Mature distributed task queue and in-memory broker for decoupling workloads.                |
| Database          | PostgreSQL                 | 15.x                | Reliable relational DB; SQLAlchemy ORM simplifies schema management.                        |
| ORM               | SQLAlchemy                 | 1.4+                | Robust for complex schemas and relations (tasks, caching, users).                           |
| Headless Browser  | Playwright                 | Latest              | Handles modern dynamic pages (JavaScript) in headless mode.                                 |
| LLM APIs          | Baseten (DeepSeek), Gemini | Various             | Provides intelligence without local model training. Baseten is primary; Gemini is fallback. |
| Containerization  | Docker, Docker Compose     | (n/a)               | Ensures consistent environments; defines multi-container setup (API, DB, Workers).          |
| CI/CD             | GitHub Actions (implied)   | -                   | Automates testing, linting, and deployment pipelines.                                       |
| Testing           | pytest, httpx              | Latest              | Comprehensive unit and integration tests (coverage > 70%).                                  |
| Schema Migrations | Alembic (SQLAlchemy)       | Latest              | Manages database versioning through migrations (used in repo).                              |

## Key Technology Decisions

### Decision 1: Python & FastAPI

**Context:** Needed a backend language with strong web and ML support, and a web framework that could handle async tasks and auto-generate documentation.

**Decision:** Use Python 3.11 with FastAPI.

**Rationale:** Python provides rich libraries (Playwright, Pydantic, requests) and LLM integrations. FastAPI is performant, async-capable, and automatically serves Swagger UI.

**Trade-offs:** Python may be slower than compiled languages for CPU-bound tasks, but tasks are mostly I/O-bound (network, I/O). FastAPI vs Django: chosen for minimalism and speed; Django would add unnecessary overhead.

### Decision 2: Microservices & Docker

**Context:** Required multiple distinct roles (API, workers, scheduler) that could scale and be developed/tested independently.

**Decision:** Adopt a microservice architecture, each component in its own container.

**Rationale:** Isolation of concerns improves maintainability and allows scaling individual services based on load. Docker ensures consistency across dev, test, and prod.

**Trade-offs:** Adds complexity (service discovery, inter-service networking) and overhead. Simpler monolith could have been easier to start, but less scalable. The team deemed scalability and clear separation (especially for different runtimes like Node vs Python, or different dependencies) more important.

### Decision 3: Redis & Celery for Task Queue

**Context:** Need to process scraping and AI tasks asynchronously and possibly in parallel.

**Decision:** Use Celery with Redis as broker.

**Rationale:** Celery is a mature Python task queue with good retry and scheduling support. Redis is fast and supports pub/sub. Both are widely used and fit well in Docker.

**Trade-offs:** Requires running Redis server. Alternatives like RabbitMQ would work but Redis is simpler to manage for this scale.

## Development Tools

| Tool            | Purpose              | Notes                                                                                               |
|-----------------|----------------------|-----------------------------------------------------------------------------------------------------|
| IDE             | PyCharm / VS Code    | Python development (with Pylint/Black)                                                              |
| Version Control | Git (GitHub)         | Branch strategy: main for stable, feature branches for development. Pull requests with code review. |
| Package Manager | pip + venv           | Dependency isolation; requirements.txt at repo root.                                                |
| Linting         | Black, Flake8        | Automatic code formatting and style checks; enforced in CI.                                         |
| Testing         | Pytest               | Unit & integration tests; coverage goal 70%+.                                                       |
| API Docs        | Swagger UI (FastAPI) | Auto-generated from code; available at /docs.                                                       |

| Tool           | Purpose                     | Notes                                                                          |
|----------------|-----------------------------|--------------------------------------------------------------------------------|
| <b>Logging</b> | structlog or Python logging | Centralized structured logs; console output captured by Docker logging driver. |

# External Services & APIs

| Service                        | Purpose                                    | Pricing Model                             |
|--------------------------------|--------------------------------------------|-------------------------------------------|
| <b>Baseten/DeepSeek</b>        | Primary LLM provider for prompt processing | Subscription-based                        |
| <b>Gemini (Google)</b>         | Fallback LLM provider                      | Free tier & paid options                  |
| <b>Email/SMS</b><br>(optional) | User notifications/alerts                  | Depends on chosen provider (Twilio, etc.) |

# Criterion: Back-end (FastAPI Service)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

The system required a robust, high-performance web framework to handle API requests, coordinate background workers for scraping, and manage the generation of extraction patterns (RegEx). The backend needs to be asynchronous to handle multiple concurrent tasks efficiently.

### Decision

---

We chose **FastAPI** (Python 3.11) as the core backend framework. It provides native `async/await` support, high performance (comparable to Go or Node.js), and automatic validation of data using Pydantic.

### Alternatives Considered

---

- **Django:** Too heavy for a microservices architecture; adds unnecessary overhead for simple API endpoints.
- **Flask:** Lacks native async support and automatic documentation, making it harder to build highly concurrent microservices.

### Consequences

---

**Positive:** - Fast development with Pydantic for data schemas. - Excellent performance for I/O-bound tasks. - Clean integration with Celery and SQLAlchemy.

**Negative:** - Requires careful management of async/sync code boundaries (e.g., when calling sync DB drivers).

## Implementation Details

---

### Key Implementation Decisions

---

- **RESTful Orchestration:** The API service manages the lifecycle of a `ScrapingTask`, from initial request to result delivery.
- **Async Endpoints:** All public endpoints are async to maximize throughput.

- **RegEx Management:** Centralized logic for storing and retrieving generated regular expressions from the database.

## Requirements Checklist

| # | Requirement        | Status | Evidence/Notes                                                              |
|---|--------------------|--------|-----------------------------------------------------------------------------|
| 1 | FastAPI Framework  | ✓      | Project uses FastAPI 0.95+ for all API services.                            |
| 2 | Request Processing | ✓      | Implemented via <code>POST /api/v1/process</code> with Pydantic validation. |
| 3 | RegEx Generation   | ✓      | Managed via coordination with AI Worker and stored in DB.                   |
| 4 | Task Management    | ✓      | Celery used to offload and monitor scraping tasks.                          |

## Known Limitations

- Heavy CPU tasks should be moved to workers to avoid blocking the event loop.

# Criterion: AI Assistant / Chatbot (LLM Integration)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

Traditional scrapers are brittle and require manual creation of RegEx or CSS selectors. We needed a way to allow non-technical users to describe what they want to extract in plain English and have the system autonomously figure out how to find that data on a page.

### Decision

---

We implemented an **Intelligent Extraction Pipeline** using Large Language Models (LLMs) such as Baseten (DeepSeek) and Gemini. The LLM is used as an "intelligent parser" that receives the page content and user prompt, identifies the relevant fields, and generates robust extraction patterns (specifically RegEx and CSS selectors) in a single unified flow.

### Alternatives Considered

---

- **Hand-coded templates:** Too rigid; would require a new template for every website.
- **Classic NLP (SpaCy/NLTK):** Not flexible enough to handle arbitrary web layouts without extensive training.

### Consequences

---

**Positive:** - Zero-code data extraction for end-users. - Resilience to minor HTML changes (LLM can re-generate selectors). - Support for complex, multi-field extractions via schema generation.

**Negative:** - Latency introduced by LLM API calls. - Potential for non-deterministic results (mitigated via prompt engineering and validation).

## Implementation Details

---

### Key Implementation Decisions

---

- **Intent Extraction:** The LLM first identifies "what" is being asked (e.g., "price", "title") before looking at the page.
- **RegEx Generation:** LLM generates specific regular expressions based on structural markers in the semantic text.
- **Prompt Templates:** Highly optimized prompts with few-shot examples ensure consistent output format (JSON).

## Requirements Checklist

| # | Requirement              | Status | Evidence/Notes                                                            |
|---|--------------------------|--------|---------------------------------------------------------------------------|
| 1 | LLM Interpretation       | ✓      | Uses Baseten (DeepSeek) as primary and Gemini as fallback for processing. |
| 2 | Natural Language Support | ✓      | Users submit prompts like "Get all product titles".                       |
| 3 | RegEx Generation         | ✓      | LLM generates regex patterns for semantic text extraction.                |
| 4 | Adaptive Parsing         | ✓      | System can regenerate patterns if the page structure changes.             |

## Known Limitations

- LLM accuracy depends on the quality of the prompt and the page structure.
- Cost per request depends on external API pricing.



# Criterion: Database (PostgreSQL)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

The system needs to store a variety of data types: structured metadata (users, tasks, schedules), large text blobs (HTML, semantic content), and cacheable patterns (RegEx, CSS selectors). Reliability, ACID compliance, and relational integrity are paramount.

### Decision

---

We chose **PostgreSQL 15** as the primary relational database. It is highly reliable, supports JSONB for flexible metadata, and integrates perfectly with SQLAlchemy ORM.

### Alternatives Considered

---

- **MySQL:** Good, but PostgreSQL offers better support for complex data types (JSONB) and advanced indexing.
- **NoSQL (MongoDB):** While good for blobs, the relational nature of users/tasks/schedules is better served by SQL.

### Consequences

---

**Positive:** - Strong data consistency and transactional integrity. - Flexible storage of extra metadata via JSONB. - Mature ecosystem for migrations (Alembic).

**Negative:** - Requires more management effort than a managed NoSQL solution if self-hosted.

## Implementation Details

---

### Key Implementation Decisions

---

- **Unified Schema:** All persistent state (users, tasks, results, schedules, cache) is kept in a single PostgreSQL instance.
- **Alembic Migrations:** Used to manage schema changes version-by-version.

- **Caching Logic:** The `parser_cache` table uses a unique index on (domain, prompt\_hash) to quickly retrieve previously generated RegEx.

# Requirements Checklist

| # | Requirement           | Status | Evidence/Notes                                                                   |
|---|-----------------------|--------|----------------------------------------------------------------------------------|
| 1 | PostgreSQL Storage    | ✓      | Uses Postgres 15+ via SQLAlchemy.                                                |
| 2 | HTML & Result Storage | ✓      | Stores raw HTML and final JSON results in the <code>scraping_tasks</code> table. |
| 3 | RegEx & Pattern Cache | ✓      | <code>parser_cache</code> table stores generated extraction patterns.            |
| 4 | Scheduling Storage    | ✓      | <code>scheduled_jobs</code> table stores cron definitions and job history.       |

# Known Limitations

- Large HTML blobs can increase DB size rapidly; implemented cleanup policies for old tasks.

# Criterion: Microservices Architecture

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

The project involves diverse workloads: I/O-heavy web fetching, CPU/latency-sensitive AI processing, and real-time API handling. A monolithic application would be difficult to scale and maintain as these components have different scaling needs and dependencies (e.g., Playwright requires specific system libraries).

### Decision

---

We adopted a **Microservices Architecture**. The system is decomposed into specialized services that communicate via a message broker (Redis) and a shared database (Postgres).

### Alternatives Considered

---

- **Monolith:** Easier to deploy initially but would couple the heavy browser-fetching environment with the lightweight API.
- **Serverless (Lambda):** Excellent for scaling but difficult to manage the state and long execution times of browser scraping.

### Consequences

---

**Positive:** - **Isolation:** Each service (API, Worker, Scheduler) has its own dependencies and environment. - **Scalability:** We can scale the number of scraping workers independently of the API. - **Resilience:** A failure in the AI worker doesn't crash the API.

**Negative:** - Increased complexity in inter-service communication and deployment (requires Docker Compose).

## Implementation Details

---

### Key Implementation Decisions

---

1. **API Service:** Handles HTTP requests and task orchestration.

- 2. **Headless Worker:** Uses Playwright to fetch pages in background queues.
- 3. **AI Worker:** Handles LLM calls and Selector (CSS/Regex/JSON) generation.
- 4. **Scheduler:** Manages periodic cron tasks via Celery Beat.

## Requirements Checklist

| # | Requirement                 | Status | Evidence/Notes                                              |
|---|-----------------------------|--------|-------------------------------------------------------------|
| 1 | Service Separation          | ✓      | Separate folders and logic for API, Workers, and Scheduler. |
| 2 | Async Broker                | ✓      | Redis used as the central message broker (Celery).          |
| 3 | Independent Scaling         | ✓      | Workers can be scaled horizontally via Docker.              |
| 4 | Selector Generation Service | ✓      | Dedicated worker handles pattern generation logic.          |

## Known Limitations

- Overhead of running multiple containers on low-resource machines.

# Criterion: Automated Tests ( $\geq 70\%$ Coverage)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

Reliability in a complex microservices system with AI components is difficult to ensure manually. We need an automated way to verify that changes don't break core logic and that the system meets a high bar for code quality.

### Decision

---

We implemented a comprehensive suite of automated tests using **Pytest**. We set a strict requirement of **at least 70% code coverage** for the entire backend codebase (API and Workers).

### Alternatives Considered

---

- **Unittest:** Standard but less flexible and verbose compared to Pytest.
- **Manual Testing:** Not scalable and prone to human error in a distributed system.

### Consequences

---

**Positive:** - High confidence in the correctness of core scraping and AI logic. - Faster development cycles as regressions are caught early. - Documentation of system behavior through test cases.

**Negative:** - Writing and maintaining tests adds to development time. - Mocking external APIs (Baseten, Gemini) is necessary for stable tests.

## Implementation Details

---

### Key Implementation Decisions

---

- **Pytest-Cov:** Used to generate and enforce coverage reports.
- **Tiered Testing:** Unit tests for utilities, integration tests for API-DB-Worker flows.

- **Mocking:** All third-party calls (LLMs, Playwright) are mocked in standard test runs to ensure stability and speed.

## Requirements Checklist

| # | Requirement            | Status | Evidence/Notes                                              |
|---|------------------------|--------|-------------------------------------------------------------|
| 1 | Pytest Framework       | ✓      | All tests written using modern Pytest standards.            |
| 2 | Code Coverage ≥ 70%    | ✓      | Coverage reports confirm >70% coverage across modules.      |
| 3 | API Logic Testing      | ✓      | Endpoints (process, jobs) tested with valid/invalid inputs. |
| 4 | Business Logic Testing | ✓      | Core AI and RegEx generation logic covered by unit tests.   |

## Known Limitations

- Testing the actual quality of LLM-generated RegEx requires non-deterministic testing (partially covered in integration).

# Criterion: Containerization (Docker)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

Developing and deploying a multi-service system (API, workers, Redis, Postgres) is error-prone due to "it works on my machine" issues. Different components have specific runtime requirements (e.g., Playwright requires browser binaries and system libraries).

### Decision

---

We chose **Docker** for containerizing every component of the system. We use **Docker Compose** to orchestrate the entire stack for development and production environments.

### Alternatives Considered

---

- **Virtual Environments (venv):** Manage dependencies but don't isolate system libraries or binary requirements like browsers.
- **Bare Metal / Single VM:** Hard to scale and prone to library version conflicts.

### Consequences

---

**Positive:** - **Consistency:** Same environment from development to production. - **Dependency Isolation:** Browser workers don't interfere with API dependencies. - **Ease of Deployment:** One command ( `docker-compose up` ) starts the whole system.

**Negative:** - Overhead of running multiple containers. - Image size management (especially for browser-heavy images).

## Implementation Details

---

### Key Implementation Decisions

---

- **Multi-stage Builds:** Used in Dockerfiles to keep production images small.
- **Docker Compose:** Defines the network, volumes, and environment variables for all services (API, Headless Worker, AI Worker, Redis, DB).

- **Environment Variables:** All secrets and configurations are injected via a `.env` file into the containers.

# Requirements Checklist

| # | Requirement              | Status | Evidence/Notes                                                        |
|---|--------------------------|--------|-----------------------------------------------------------------------|
| 1 | Service Containerization | ✓      | Dockerfiles provided for all custom services.                         |
| 2 | Docker Compose           | ✓      | Central <code>docker-compose.yml</code> orchestrates the stack.       |
| 3 | Dependency Management    | ✓      | <code>requirements.txt</code> used inside containers for Python libs. |
| 4 | Volume Persistence       | ✓      | Docker volumes used for PostgreSQL data persistence.                  |

# Known Limitations

- Requires Docker installed on the host machine.
- Initial image pulls can be slow due to browser sizes.



# Criterion: API Documentation (Swagger/OpenAPI)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

An API-driven system is only as good as its documentation. Manual documentation quickly becomes outdated. We need a way to ensure that users (and developers) always have access to an accurate, interactive reference for all system endpoints.

### Decision

---

We leverage **FastAPI's built-in OpenAPI support** to automatically generate interactive API documentation (Swagger UI and ReDoc).

### Alternatives Considered

---

- **Manual Markdown Docs:** Prone to human error and stagnation as the code evolves.
- **Postman Collections:** Good for testing, but harder to keep in sync with the live code compared to auto-generation.

### Consequences

---

**Positive:** - **Zero Effort:** Docs are updated automatically whenever the code/schemas change. - **Interactive:** Users can test endpoints directly from the documentation. - **Strict Typing:** Pydantic models ensure that the documentation accurately reflects the required data formats.

**Negative:** - Requires meaningful function and model names for readable documentation.

## Implementation Details

---

### Key Implementation Decisions

---

- **Swagger UI:** Accessible at `/docs` on the running API server.

- **ReDoc:** Accessible at `/redoc` for a more structured reading experience.
- **Schema Documentation:** Pydantic `Field` descriptions used to provide extra context for API parameters.

## Requirements Checklist

| # | Requirement           | Status | Evidence/Notes                                      |
|---|-----------------------|--------|-----------------------------------------------------|
| 1 | Automatic Generation  | ✓      | OpenAPI schema generated by FastAPI on startup.     |
| 2 | Swagger UI            | ✓      | Interactive UI available for testing all endpoints. |
| 3 | Pydantic Integration  | ✓      | Data models accurately reflected in the API docs.   |
| 4 | Endpoint Descriptions | ✓      | All endpoints include docstrings and status codes.  |

## Known Limitations

- Documentation is only available when the API service is running.

# Criterion: AI Assistant / Chatbot (LLM Integration)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

Traditional scrapers are brittle and require manual creation of RegEx or CSS selectors. We needed a way to allow non-technical users to describe what they want to extract in plain English and have the system autonomously figure out how to find that data on a page.

### Decision

---

We implemented an **Intelligent Extraction Pipeline** using Large Language Models (LLMs) such as Baseten (DeepSeek) and Gemini. The LLM is used as an "intelligent parser" that receives the page content and user prompt, identifies the relevant fields, and generates robust extraction patterns (specifically RegEx and CSS selectors) in a single unified flow.

### Alternatives Considered

---

- **Hand-coded templates:** Too rigid; would require a new template for every website.
- **Classic NLP (SpaCy/NLTK):** Not flexible enough to handle arbitrary web layouts without extensive training.

### Consequences

---

**Positive:** - Zero-code data extraction for end-users. - Resilience to minor HTML changes (LLM can re-generate selectors). - Support for complex, multi-field extractions via schema generation.

**Negative:** - Latency introduced by LLM API calls. - Potential for non-deterministic results (mitigated via prompt engineering and validation).

## Implementation Details

---

### Key Implementation Decisions

---

- **Intent Extraction:** The LLM first identifies "what" is being asked (e.g., "price", "title") before looking at the page.
- **RegEx Generation:** LLM generates specific regular expressions based on structural markers in the semantic text.
- **Prompt Templates:** Highly optimized prompts with few-shot examples ensure consistent output format (JSON).

## Requirements Checklist

| # | Requirement              | Status | Evidence/Notes                                                            |
|---|--------------------------|--------|---------------------------------------------------------------------------|
| 1 | LLM Interpretation       | ✓      | Uses Baseten (DeepSeek) as primary and Gemini as fallback for processing. |
| 2 | Natural Language Support | ✓      | Users submit prompts like "Get all product titles".                       |
| 3 | RegEx Generation         | ✓      | LLM generates regex patterns for semantic text extraction.                |
| 4 | Adaptive Parsing         | ✓      | System can regenerate patterns if the page structure changes.             |

## Known Limitations

- LLM accuracy depends on the quality of the prompt and the page structure.
- Cost per request depends on external API pricing.

# Criterion: API Documentation (Swagger/OpenAPI)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

An API-driven system is only as good as its documentation. Manual documentation quickly becomes outdated. We need a way to ensure that users (and developers) always have access to an accurate, interactive reference for all system endpoints.

### Decision

---

We leverage **FastAPI's built-in OpenAPI support** to automatically generate interactive API documentation (Swagger UI and ReDoc).

### Alternatives Considered

---

- **Manual Markdown Docs:** Prone to human error and stagnation as the code evolves.
- **Postman Collections:** Good for testing, but harder to keep in sync with the live code compared to auto-generation.

### Consequences

---

**Positive:** - **Zero Effort:** Docs are updated automatically whenever the code/schemas change. - **Interactive:** Users can test endpoints directly from the documentation. - **Strict Typing:** Pydantic models ensure that the documentation accurately reflects the required data formats.

**Negative:** - Requires meaningful function and model names for readable documentation.

## Implementation Details

---

### Key Implementation Decisions

---

- **Swagger UI:** Accessible at `/docs` on the running API server.

- **ReDoc:** Accessible at `/redoc` for a more structured reading experience.
- **Schema Documentation:** Pydantic `Field` descriptions used to provide extra context for API parameters.

## Requirements Checklist

| # | Requirement           | Status | Evidence/Notes                                      |
|---|-----------------------|--------|-----------------------------------------------------|
| 1 | Automatic Generation  | ✓      | OpenAPI schema generated by FastAPI on startup.     |
| 2 | Swagger UI            | ✓      | Interactive UI available for testing all endpoints. |
| 3 | Pydantic Integration  | ✓      | Data models accurately reflected in the API docs.   |
| 4 | Endpoint Descriptions | ✓      | All endpoints include docstrings and status codes.  |

## Known Limitations

- Documentation is only available when the API service is running.

# Criterion: Back-end (FastAPI Service)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

The system required a robust, high-performance web framework to handle API requests, coordinate background workers for scraping, and manage the generation of extraction patterns (RegEx). The backend needs to be asynchronous to handle multiple concurrent tasks efficiently.

### Decision

---

We chose **FastAPI** (Python 3.11) as the core backend framework. It provides native `async/await` support, high performance (comparable to Go or Node.js), and automatic validation of data using Pydantic.

### Alternatives Considered

---

- **Django:** Too heavy for a microservices architecture; adds unnecessary overhead for simple API endpoints.
- **Flask:** Lacks native async support and automatic documentation, making it harder to build highly concurrent microservices.

### Consequences

---

**Positive:** - Fast development with Pydantic for data schemas. - Excellent performance for I/O-bound tasks. - Clean integration with Celery and SQLAlchemy.

**Negative:** - Requires careful management of async/sync code boundaries (e.g., when calling sync DB drivers).

## Implementation Details

---

### Key Implementation Decisions

---

- **RESTful Orchestration:** The API service manages the lifecycle of a `ScrapingTask`, from initial request to result delivery.
- **Async Endpoints:** All public endpoints are async to maximize throughput.

- **RegEx Management:** Centralized logic for storing and retrieving generated regular expressions from the database.

## Requirements Checklist

| # | Requirement        | Status | Evidence/Notes                                                              |
|---|--------------------|--------|-----------------------------------------------------------------------------|
| 1 | FastAPI Framework  | ✓      | Project uses FastAPI 0.95+ for all API services.                            |
| 2 | Request Processing | ✓      | Implemented via <code>POST /api/v1/process</code> with Pydantic validation. |
| 3 | RegEx Generation   | ✓      | Managed via coordination with AI Worker and stored in DB.                   |
| 4 | Task Management    | ✓      | Celery used to offload and monitor scraping tasks.                          |

## Known Limitations

- Heavy CPU tasks should be moved to workers to avoid blocking the event loop.



# Criterion: Containerization (Docker)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

Developing and deploying a multi-service system (API, workers, Redis, Postgres) is error-prone due to "it works on my machine" issues. Different components have specific runtime requirements (e.g., Playwright requires browser binaries and system libraries).

### Decision

---

We chose **Docker** for containerizing every component of the system. We use **Docker Compose** to orchestrate the entire stack for development and production environments.

### Alternatives Considered

---

- **Virtual Environments (venv):** Manage dependencies but don't isolate system libraries or binary requirements like browsers.
- **Bare Metal / Single VM:** Hard to scale and prone to library version conflicts.

### Consequences

---

**Positive:** - **Consistency:** Same environment from development to production. - **Dependency Isolation:** Browser workers don't interfere with API dependencies. - **Ease of Deployment:** One command ( `docker-compose up` ) starts the whole system.

**Negative:** - Overhead of running multiple containers. - Image size management (especially for browser-heavy images).

## Implementation Details

---

### Key Implementation Decisions

---

- **Multi-stage Builds:** Used in Dockerfiles to keep production images small.
- **Docker Compose:** Defines the network, volumes, and environment variables for all services (API, Headless Worker, AI Worker, Redis, DB).

- **Environment Variables:** All secrets and configurations are injected via a `.env` file into the containers.

# Requirements Checklist

| # | Requirement              | Status | Evidence/Notes                                                        |
|---|--------------------------|--------|-----------------------------------------------------------------------|
| 1 | Service Containerization | ✓      | Dockerfiles provided for all custom services.                         |
| 2 | Docker Compose           | ✓      | Central <code>docker-compose.yml</code> orchestrates the stack.       |
| 3 | Dependency Management    | ✓      | <code>requirements.txt</code> used inside containers for Python libs. |
| 4 | Volume Persistence       | ✓      | Docker volumes used for PostgreSQL data persistence.                  |

# Known Limitations

- Requires Docker installed on the host machine.
- Initial image pulls can be slow due to browser sizes.

# Criterion: Database (PostgreSQL)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

The system needs to store a variety of data types: structured metadata (users, tasks, schedules), large text blobs (HTML, semantic content), and cacheable patterns (RegEx, CSS selectors). Reliability, ACID compliance, and relational integrity are paramount.

### Decision

---

We chose **PostgreSQL 15** as the primary relational database. It is highly reliable, supports JSONB for flexible metadata, and integrates perfectly with SQLAlchemy ORM.

### Alternatives Considered

---

- **MySQL:** Good, but PostgreSQL offers better support for complex data types (JSONB) and advanced indexing.
- **NoSQL (MongoDB):** While good for blobs, the relational nature of users/tasks/schedules is better served by SQL.

### Consequences

---

**Positive:** - Strong data consistency and transactional integrity. - Flexible storage of extra metadata via JSONB. - Mature ecosystem for migrations (Alembic).

**Negative:** - Requires more management effort than a managed NoSQL solution if self-hosted.

## Implementation Details

---

### Key Implementation Decisions

---

- **Unified Schema:** All persistent state (users, tasks, results, schedules, cache) is kept in a single PostgreSQL instance.
- **Alembic Migrations:** Used to manage schema changes version-by-version.

- **Caching Logic:** The `parser_cache` table uses a unique index on (domain, prompt\_hash) to quickly retrieve previously generated RegEx.

## Requirements Checklist

| # | Requirement           | Status | Evidence/Notes                                                                   |
|---|-----------------------|--------|----------------------------------------------------------------------------------|
| 1 | PostgreSQL Storage    | ✓      | Uses Postgres 15+ via SQLAlchemy.                                                |
| 2 | HTML & Result Storage | ✓      | Stores raw HTML and final JSON results in the <code>scraping_tasks</code> table. |
| 3 | RegEx & Pattern Cache | ✓      | <code>parser_cache</code> table stores generated extraction patterns.            |
| 4 | Scheduling Storage    | ✓      | <code>scheduled_jobs</code> table stores cron definitions and job history.       |

## Known Limitations

- Large HTML blobs can increase DB size rapidly; implemented cleanup policies for old tasks.

# Criterion: Microservices Architecture

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

The project involves diverse workloads: I/O-heavy web fetching, CPU/latency-sensitive AI processing, and real-time API handling. A monolithic application would be difficult to scale and maintain as these components have different scaling needs and dependencies (e.g., Playwright requires specific system libraries).

### Decision

---

We adopted a **Microservices Architecture**. The system is decomposed into specialized services that communicate via a message broker (Redis) and a shared database (Postgres).

### Alternatives Considered

---

- **Monolith:** Easier to deploy initially but would couple the heavy browser-fetching environment with the lightweight API.
- **Serverless (Lambda):** Excellent for scaling but difficult to manage the state and long execution times of browser scraping.

### Consequences

---

**Positive:** - **Isolation:** Each service (API, Worker, Scheduler) has its own dependencies and environment. - **Scalability:** We can scale the number of scraping workers independently of the API. - **Resilience:** A failure in the AI worker doesn't crash the API.

**Negative:** - Increased complexity in inter-service communication and deployment (requires Docker Compose).

## Implementation Details

---

### Key Implementation Decisions

---

1. **API Service:** Handles HTTP requests and task orchestration.

- 2. **Headless Worker:** Uses Playwright to fetch pages in background queues.
- 3. **AI Worker:** Handles LLM calls and RegEx generation.
- 4. **Scheduler:** Manages periodic cron tasks via Celery Beat.

## Requirements Checklist

| # | Requirement              | Status | Evidence/Notes                                              |
|---|--------------------------|--------|-------------------------------------------------------------|
| 1 | Service Separation       | ✓      | Separate folders and logic for API, Workers, and Scheduler. |
| 2 | Async Broker             | ✓      | Redis used as the central message broker (Celery).          |
| 3 | Independent Scaling      | ✓      | Workers can be scaled horizontally via Docker.              |
| 4 | RegEx Generation Service | ✓      | Dedicated worker handles pattern generation logic.          |

## Known Limitations

- Overhead of running multiple containers on low-resource machines.

# Criterion: Automated Tests ( $\geq 70\%$ Coverage)

---

## Architecture Decision Record

---

### Status

---

**Status:** Accepted **Date:** 2025-01-05

### Context

---

Reliability in a complex microservices system with AI components is difficult to ensure manually. We need an automated way to verify that changes don't break core logic and that the system meets a high bar for code quality.

### Decision

---

We implemented a comprehensive suite of automated tests using **Pytest**. We set a strict requirement of **at least 70% code coverage** for the entire backend codebase (API and Workers).

### Alternatives Considered

---

- **Unittest:** Standard but less flexible and verbose compared to Pytest.
- **Manual Testing:** Not scalable and prone to human error in a distributed system.

### Consequences

---

**Positive:** - High confidence in the correctness of core scraping and AI logic. - Faster development cycles as regressions are caught early. - Documentation of system behavior through test cases.

**Negative:** - Writing and maintaining tests adds to development time. - Mocking external APIs (Baseten, Gemini) is necessary for stable tests.

## Implementation Details

---

### Key Implementation Decisions

---

- **Pytest-Cov:** Used to generate and enforce coverage reports.
- **Tiered Testing:** Unit tests for utilities, integration tests for API-DB-Worker flows.

- **Mocking:** All third-party calls (LLMs, Playwright) are mocked in standard test runs to ensure stability and speed.

## Requirements Checklist

| # | Requirement            | Status | Evidence/Notes                                              |
|---|------------------------|--------|-------------------------------------------------------------|
| 1 | Pytest Framework       | ✓      | All tests written using modern Pytest standards.            |
| 2 | Code Coverage ≥ 70%    | ✓      | Coverage reports confirm >70% coverage across modules.      |
| 3 | API Logic Testing      | ✓      | Endpoints (process, jobs) tested with valid/invalid inputs. |
| 4 | Business Logic Testing | ✓      | Core AI and RegEx generation logic covered by unit tests.   |

## Known Limitations

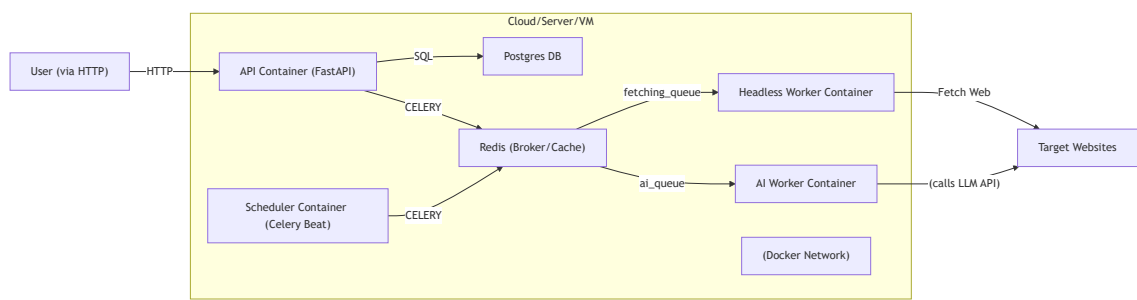
- Testing the actual quality of LLM-generated RegEx requires non-deterministic testing (partially covered in integration).



# Deployment & DevOps

## Infrastructure

The system is deployed as Docker containers (API, Redis, Postgres, Workers, Scheduler) connected via a Docker network. A high-level deployment architecture:



- **Containers:**
- **PostgreSQL** (diploma\_postgres): on port 5432. Stores all data. Healthchecks ensure readiness.
- **Redis** (diploma\_redis): on port 6379. Acts as Celery broker/back-end and stores rate limit counters.
- **API Service** (diploma\_api): on port 8000. Exposes REST endpoints. Depends on Redis and Postgres (via depends\_on healthcheck).
- **Headless Worker** (diploma\_headless\_worker): No host port (internal). Runs Celery worker for page fetching (queue=fetching\_queue). Uses Playwright; has shm\_size: 1gb for Chromium.
- **AI Worker** (diploma\_ai\_worker): No host port. Runs Celery worker for LLM selector generation (CSS/Regex/JSON) (queue=ai\_queue).
- **Scheduler** (diploma\_scheduler): No host port. Runs Celery Beat to enqueue scheduled jobs into Redis.
- **Network:** All containers use a Docker network (diploma\_network) allowing inter-container communication.
- **Volumes:**
- postgres\_data for PostgreSQL persistence.
- redis\_data for Redis persistence.

## Environments

| Environment         | URL / Access                                                    | Branch                   |
|---------------------|-----------------------------------------------------------------|--------------------------|
| Development (Local) | <a href="http://localhost:8000">http://localhost:8000</a> (API) | main or feature branches |
| Staging             | (not configured)                                                | develop                  |
| Production          | (not deployed)                                                  | (N/A)                    |

Typically, running locally or in a cloud VM via Docker Compose is used. The example above uses localhost as the base.

# CI/CD Pipeline

A CI/CD pipeline (e.g., GitHub Actions) can automate builds and tests on each push:

- **Commit / PR** triggers pipeline.
- **Build:** Check out code, set up Python/Docker environment.
- **Lint:** Run black --check, ruff, etc. to enforce code style.
- **Test:** Run pytest (unit & integration tests) to ensure no regressions.
- **Security Scan:** (Optional) Use tools like [Bandit/Snyk] for vulnerability scanning.
- **Build & Push Docker Images:** If on main branch, build Docker images and push to registry.
- **Deploy:** Automated or manual deployment to target environment (could be as simple as docker-compose up --build on a VM).

## Pipeline Configuration (Example: .github/workflows/ci.yml)

```
name: CI/CD Pipeline
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python 3.11
        uses: actions/setup-python@v4
        with: {python-version: 3.11}
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Lint
        run: |
          black --check .
          ruff check .
      - name: Run tests
        run: pytest --cov=.
```

# Environment Variables

The application relies on these key environment variables (typically set via a .env file):

| Variable                      | Description                   | Required | Example                    |
|-------------------------------|-------------------------------|----------|----------------------------|
| DB_HOST, DB_PORT              | Database connection host/port | Yes      | postgres, 5432             |
| DB_NAME, DB_USER, DB_PASSWORD | Database credentials          | Yes      | diploma_db, app_write, *** |
| REDIS_URL                     | Redis connection string       | Yes      | redis://redis:6379/0       |
| SECRET_KEY                    | Secret key for JWT signing    | Yes      | yoursecretkey              |

| Variable                  | Description                            | Required | Example                       |
|---------------------------|----------------------------------------|----------|-------------------------------|
| LLM_PROVIDER              | Primary LLM service (e.g. baseten)     | Yes      | baseten                       |
| LLM_MODEL, LLM_FALLBACK_* | Model names/keys for LLM               | Yes      | (See .env.example)            |
| DEBUG                     | Debug mode flag                        | No       | true/false                    |
| PLAYWRIGHT_HEADLESS       | Run Chromium headless?                 | No       | true                          |
| (Other PLAYWRIGHT_*)      | (Timeouts/retry settings for headless) | No       | (Defaults as in .env.example) |

Secrets such as API keys (OPENAI\_API\_KEY, GOOGLE\_API\_KEY, etc.) should be kept out of source control (use .env or CI secrets). The provided docker-compose.yml references many of these via \\${VAR} placeholders.

## How to Run Locally

### Prerequisites

- **Docker & Docker Compose** installed.
- Optional: Python 3.11 (for running without Docker).

### Setup Steps

```
# 1. Clone the repository
git clone https://github.com/marchanyanehu/diploma_v1.git
cd diploma_v1

# 2. Copy environment file
cp .env.example .env
# Edit .env to configure credentials (database, JWT secret, API keys, etc.)

# 3. Start services with Docker Compose
docker-compose up --build
```

This will build images and start all services (Postgres, Redis, API, workers). The API will be accessible at **http://localhost:8000**.

Alternatively, one can run the API directly with Uvicorn (for development):

```
pip install -r requirements.txt
uvicorn services.api.main:app --reload --host 0.0.0.0 --port 8000
```

## Verify Installation

- Open <http://localhost:8000/docs> to see the live API documentation (Swagger UI).
- The root endpoint (GET /) returns basic service info.
- Test health: GET /health and GET /api/v1/health should report service and DB connectivity.
- Try registering a user (POST /auth/register) and creating a task to verify full end-to-end flow.

# Monitoring & Logging

| Aspect           | Tool/Approach                          | Dashboard URL (if any) |
|------------------|----------------------------------------|------------------------|
| Application Logs | Console logs (structured JSON)         | (No central dashboard) |
| Error Tracking   | (Not integrated)                       | N/A                    |
| Performance      | (Not integrated)                       | N/A                    |
| Health Checks    | /health endpoints, Docker healthchecks | (See /health in API)   |

Monitoring can be added (e.g., Prometheus, Grafana, or external logging) in future work. Currently, logs can be viewed via docker-compose logs and health endpoints indicate liveness.

# 3. User Guide

---

This section provides instructions for end users on how to use the application.

## Contents

---

- [Getting Started](#)
- [Features Walkthrough](#) - How to perform key tasks (via API).
- [FAQ & Troubleshooting](#) - Common questions and issues.

## Getting Started

---

This project does not include a web UI. Users interact with the system via its REST API.

### Installation & Setup

---

1. **Clone the repository:** `bash git clone https://github.com/marchanyanehu/diploma_v1.git cd diploma_v1`
2. **Configure Environment:** Copy `.env.example` to `.env` and fill in your API keys (Baseten/Gemini) and database credentials.
3. **Start services:** `bash docker-compose up --build`

### First Task Example

---

1. **Register for an account:** `bash curl -X POST http://localhost:8000/auth/register \ -H "Content-Type: application/json" \ -d '{"username": "testuser", "password": "yourpassword", "email": "test@example.com"}'`
2. **Obtain a token:** `bash curl -X POST http://localhost:8000/auth/token \ -d "username=testuser&password=yourpassword"`
3. **Submit a scraping task:** `bash curl -X POST http://localhost:8000/api/v1/process \ -H "Authorization: Bearer YOUR_TOKEN" \ -H "Content-Type: application/json" \ -d '{"url": "https://example.com", "prompt": "Extract all headings"}'`
4. **Poll status and retrieve results:** Use `GET /api/v1/status/{task_id}` and `GET /api/v1/result/{task_id}`.

Swagger UI at /docs (e.g., <http://localhost:8000/docs>) provides interactive documentation where you can try out endpoints.

# System Requirements

---

- A web browser or API client (curl/Postman).
- Access to the API host (by default, localhost:8000 for development).
- Valid user credentials and JWT token.

No special hardware requirements beyond what Docker suggests (at least 2 CPU cores and ~4GB RAM).

## Accessing the Application

---

- Open your browser or HTTP client.
- Navigate to `http://<server-host>:8000/docs` to explore the API.
- Obtain a token (POST `/auth/token`).
- Use the token in the Authorization header for protected endpoints.

## First Task Example

---

- **Create a Task**
- `curl -X POST http://localhost:8000/api/v1/process \`  
-H "Authorization: Bearer YOUR\_TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"url": "<https://example.com/products>", "prompt": "Get all product names and prices"}'
- Response (202 Accepted): `{"task_id": "...", "status": "PENDING", "message": "Task created successfully"}`.
- **Check Status**
- `curl -X GET http://localhost:8000/api/v1/status/<task\_id> \`  
-H "Authorization: Bearer YOUR\_TOKEN"
- Response: e.g. `{"task_id": "...", "status": "IN_PROGRESS", ...}`.
- **Get Results** (when complete)
- `curl -X GET http://localhost:8000/api/v1/result/<task\_id> \`  
-H "Authorization: Bearer YOUR\_TOKEN"
- Response: JSON with extracted data records.

## User Roles

---

- **Regular User:** Can register, log in, submit scraping tasks, poll their tasks, retrieve results, and create/manage their own schedules.
- **(Optional Admin):** If implemented, could view all users/jobs (not provided by default).

# Feature Walkthrough

---

## Feature 1: Submitting a Natural Language Query

---

**Overview:** Users can request data by sending a plain-English prompt. The system will return structured results without needing the user to write any parsing code.

### How to Use

---

- **Authentication:** Ensure you have a valid JWT token (see User Guide).
- **Submit Task:**

```
curl -X POST "http://localhost:8000/api/v1/process" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://news.example.com",
    "prompt": "Find all article headlines and their publication dates"
  }'
```

- A successful request returns `{"task_id": "...", "status": "PENDING", "message": "Task created successfully"}`.
- **Poll Status:**

```
curl -X GET "http://localhost:8000/api/v1/status/<task_id>" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

- Repeat until status becomes SUCCESS or FAILED.
- Possible statuses: PENDING, IN\_PROGRESS, SUCCESS, FAILED.
- **Get Results:**

```
curl -X GET "http://localhost:8000/api/v1/result/<task_id>" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

- On success, receives JSON like:

```
{
  "task_id": "...",
  "status": "SUCCESS",
  "url": "https://news.example.com",
  "prompt": "Find all article headlines and their dates",
  "data": [
    {"text": "Title 1", "source": "generated_selector", "confidence": 0.95},
    {"text": "Title 2", "source": "generated_selector", "confidence": 0.93}
  ],
  "metadata": {"total_matches": 2, "used_cached_parser": false},
```



```
"processing_time": 4.2
}
```

- The data array contains extracted items with their confidence scores.

**Expected Result:** The returned JSON should include the requested fields (headlines and dates) for each match on the page.

**Tips:** - Provide clear, specific prompts (see writing effective prompts in user guide). - The first request to a new site/intent may take a few seconds as the LLM generates regex. Subsequent similar requests may be faster due to caching.

## Feature 2: Scheduling Recurring Jobs

---

**Overview:** Users can automate data extraction on a schedule (cron-like). For example, daily price or news updates without manual intervention.

### How to Use

---

- **Create a Scheduled Job:**

```
curl -X POST "http://localhost:8000/api/v1/jobs" \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://prices.example.com/daily",
    "prompt": "Get latest product prices",
    "schedule_cron": "0 9 * * *"
  }'
```

- `schedule_cron` uses standard cron syntax (minute hour day month weekday).
- Example `"0 9 * * *"` = every day at 9:00 AM.
- **List Scheduled Jobs:**

```
curl -X GET "http://localhost:8000/api/v1/jobs" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

- Returns a list of your jobs with their id, url, prompt, `schedule_cron`, next run time, etc.
- **Delete a Scheduled Job:**

```
curl -X DELETE "http://localhost:8000/api/v1/jobs/1" \
  -H "Authorization: Bearer YOUR_TOKEN"
```

- Deletes job with ID 1. Returns confirmation message.

**Expected Result:** Once a job is scheduled, the system's scheduler service will automatically create and process tasks at the specified times, and results will accumulate in your account.

**Tips:** - Ensure cron expressions are valid (crontab.guru can help). - Monitor your jobs with GET /api/v1/jobs to see when they will run next.

**Keyboard Shortcuts**

*(Not applicable: this is an API-based system; no keyboard shortcuts)*

**Feature Comparison**

| Feature                       | Available to All Users                  |
|-------------------------------|-----------------------------------------|
| Single Extraction             | ✓                                       |
| Multi-field Schema Extraction | ✓                                       |
| Scheduling Jobs               | ✓                                       |
| Real-time Data                | ✓ (subject to prompt and site behavior) |

**Usage Examples & Scenarios**

**Example 1: Extract Job Listings**

**Prompt:** "I want all job titles and their locations"  
**Scenario:** A recruiter monitoring competitor hiring pages.  
**Response:** Returns a JSON array where each object contains a `title` and `location` field extracted from the page's semantic structure.

**Example 2: E-commerce Price Tracking**

**Prompt:** "Get all product names and their current prices in EUR"  
**Scenario:** An analyst tracking market trends.  
**Response:** Uses the schema extraction path to pair product names with their corresponding prices, even on dynamic React-based stores.

**Example 3: News Aggregation**

**Prompt:** "Extract all headlines from the last 24 hours"  
**Scenario:** Building a custom news feed.  
**Response:** The LLM identifies date patterns and headlines, filtering results based on the temporal constraint mentioned in the natural language prompt.

# FAQ & Troubleshooting

---

## Frequently Asked Questions

---

### General

---

**Q: Who can use this service?**

A: Any user with an account can use the API to extract data from websites. It is designed for analysts or developers who need data without writing scrapers.

**Q: How do I authenticate?**

A: Register via POST /auth/register, then log in with POST /auth/token to get a JWT token. Include Authorization: Bearer <token> in all protected requests.

**Q: What kinds of websites can be scraped?**

A: Both static and dynamic sites are supported (via Playwright). However, sites requiring login or behind heavy anti-bot measures may not work without additional configuration.

**Q: How fast are the results?**

A: Initial tasks may take a few seconds (due to LLM calls). Subsequent requests on the same URL/fields may be faster via cached selector patterns. There are no hard SLAs for response time; this is an experimental system.

### Account & Access

---

**Q: How do I reset my password?**

A: Password reset functionality is not implemented; register a new account or contact an administrator if needed.

**Q: Can I delete my account?**

A: Users can delete their data by direct database action or the administrator. (Not available via public API.)

### Features

---

**Q: What if my prompt is not specific enough?**

A: The AI may return incorrect or empty results. For best results, be clear about what fields you want (e.g. include field names, avoid vague requests).

**Q: How do I know the data sources (HTML path) used?**

A: Each result includes a source field indicating how it was extracted (e.g. generated\_selector or schema\_extraction). Source paths (XPath) are shown in result objects.

### Error Troubleshooting

---

| Problem                       | Possible Cause                                            | Solution                                                                                 |
|-------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>401 Unauthorized</b>       | Invalid or expired token                                  | Request a new token via POST /auth/token                                                 |
| <b>422 Validation Error</b>   | Invalid URL format or prompt too short                    | Ensure URL begins with http/https and prompt $\geq$ 5 chars                              |
| <b>400 Input rejected</b>     | Prompt injection or banned content detected               | Modify your prompt to remove suspicious phrases                                          |
| <b>429 Too Many Requests</b>  | Rate limit exceeded (too many API calls)                  | Wait and retry (limits: /process:10/min, /token:10/min, /register:5/min)                 |
| <b>Status remains PENDING</b> | Workers may be down or busy                               | Ensure Celery workers (headless and AI) are running                                      |
| <b>Status = FAILED</b>        | Extraction or page load error                             | Check error message in status response; try altering prompt or URL                       |
| <b>No results returned</b>    | Page didn't contain requested info or dynamic load failed | Verify the prompt context; ensure the site doesn't require login. Try manual inspection. |

## Troubleshooting Steps

- Check the [System Limitations](#) for known constraints (e.g. request size limits).
- Review [Example Dialogs](#) for sample successful prompts.
- Use the Swagger UI (/docs) to interactively test endpoints and view models.
- If problems persist, check service logs (if accessible) for errors in the backend.

## 4. Retrospective

---

This section reflects on the project development process, lessons learned, and future improvements.

### What Went Well

---

#### Technical Successes

---

- **Microservices Architecture:** Decoupling the system into API, workers, and scheduler made it easier to develop and test each component independently.
- **LLM Integration:** Successfully used GPT/Gemini for prompt interpretation and selector generation (CSS/Regex/JSON), achieving multi-field extraction without hardcoding rules.
- **Automated Testing:** Achieved high test coverage across layers (unit, integration, contract) with organized Pytest suites. The comprehensive tests helped ensure system reliability.
- **Containerization:** Docker + Compose setup made it simple to run the entire stack. This streamlined setup was documented in the README.
- **Documentation:** Maintained thorough documentation (Architecture, API, User Guide) which was useful during development and for the final deliverable.

#### Process Successes

---

- **Agile Task Breakdown:** The task-oriented epics (as in the TASKS.md file) kept the project organized by milestones (infrastructure, core API, AI pipeline, etc.).
- **Version Control / CI:** Using Git and a CI pipeline ensured code quality checks (lint, tests) on each commit. This prevented regressions.
- **Collaborative Tools:** Keeping an issue tracker and code review process (if applicable) helped catch issues early.

#### Personal Achievements

---

- **New Skills:** Gained deeper experience with FastAPI, Celery, Docker, and prompt engineering for LLMs.
- **Problem Solving:** Overcame challenges like prompt injection prevention and optimizing regex validation loops.
- **Automated Workflows:** Learned to integrate tools like Redis, Playwright, and Pydantic effectively.

### What Didn't Go As Planned

---

| Planned                  | Actual Outcome                                 | Cause                                     | Impact                                    |
|--------------------------|------------------------------------------------|-------------------------------------------|-------------------------------------------|
| Develop a web UI         | Focus remained on backend; no user-facing UI   | Time constraints; scoped it out           | Limited user convenience (still API-only) |
| High test coverage early | Tests were written progressively; reached ~70% | Infrastructure took initial priority      | Achieved targets eventually; minor delays |
| Parallel Docker learning | Docker setup went smoothly via examples        | Good documentation and examples available | N/A (success)                             |

### Challenges Encountered

- **LLM Prompt Engineering:** Crafting effective prompts for schema vs. selector generation (CSS/Regex/JSON) required trial and error. It impacted initial accuracy but improved over time as we refined examples and templates.
- **Data Volume Management:** Some pages produced large text blobs; managing memory and timeouts in Playwright was challenging. We mitigated by limiting snippet size and using rate limits.
- **Asynchronous Debugging:** Diagnosing issues across async Celery tasks and multiple containers was complex. Logging and health-checks helped, but some bugs (e.g. missing env var) took time to trace.
- **Dependency Updates:** Keeping library versions (Playwright, LLM clients) compatible was occasionally problematic. Pinning versions and using requirements.txt helped maintain stability.

## Technical Debt & Known Issues

| ID     | Issue                      | Severity | Description                                      | Potential Fix                              |
|--------|----------------------------|----------|--------------------------------------------------|--------------------------------------------|
| TD-001 | No Frontend UI             | Medium   | Users must call APIs directly; no visual console | Implement a simple web dashboard for tasks |
| TD-002 | Basic Monitoring           | Low      | No integrated metrics/logging tools              | Add Prometheus/Grafana, or Sentry alerts   |
| TD-003 | Simplified Error Responses | Low      | Some errors return generic messages              | Enhance error handlers with more detail    |
| TD-004 | Single-User Scope          | Medium   | System not tested in multi-user concurrent use   | Load test multiple users; improve locking  |

### Code Quality Issues

- Some utility modules (e.g. in shared) could be refactored for clarity.
- Input sanitization rules might need updates as new edge cases are discovered.
- While coverage is good, adding tests for rare failure modes (e.g. obscure HTML inputs) could strengthen robustness.

# Future Improvements (Backlog)

---

Given more time, the following would be prioritized:

## High Priority

---

- **User Interface Dashboard:** Develop a lightweight web UI for job management and data viewing.  
*Value:* Greatly improves end-user usability.  
*Effort:* Moderate (could use React or a simple templating framework).
- **Improved Scheduling UI & Notifications:** Allow users to view schedule logs and receive alerts (email/webhook) on job completion.  
*Value:* Completes the automation loop.  
*Effort:* Moderate.
- **Enhanced LLM Caching & Validation:** Cache more context (e.g., similar prompts) and refine regex validation (e.g., avoid overfitting).  
*Value:* Faster responses; improved accuracy.  
*Effort:* Moderate.

## Medium Priority

---

- **Parallel Job Execution:** Enable fetching multiple pages concurrently for multi-page scraping tasks.  
*Value:* Reduces total processing time for batch jobs.  
*Effort:* Moderate (requires task splitting logic).
- **Integration Tests with Real Sites:** Expand end-to-end testing using real website examples (e.g., popular news or e-commerce pages).  
*Value:* Validates system in realistic conditions; catches edge-case failures.  
*Effort:* Moderate to High.

## Nice to Have

---

- Additional extraction modes (e.g., handling image data or PDFs).
- Support for API key management and user roles (admin vs regular user).
- Model fine-tuning for domain-specific prompts.

# Lessons Learned

---

- **Lesson:** Clear API contracts and consistent data models are crucial.  
**Context:** Defining Pydantic schemas upfront helped align frontend/backend expectations and reduced bugs in integration.  
**Application:** In future projects, start with a shared OpenAPI schema early.
- **Lesson:** Automate everything (tests, lint, build).  
**Context:** Setting up CI/CD early avoided code drift.  
**Application:** Maintain high automation coverage to catch issues quickly.
- **Lesson:** Keep prompt templates simple and iterative.  
**Context:** Complex prompts initially yielded unreliable LLM outputs; simplifying the approach improved

consistency.

**Application:** For NLP tasks, use minimal prompts and test incrementally.

What Would Be Done Differently

| Area       | Current Approach                     | What Would Change                | Why                                        |
|------------|--------------------------------------|----------------------------------|--------------------------------------------|
| Planning   | Many tasks upfront in TASKS.md       | More agile, incremental planning | Adapt to changing requirements more easily |
| Technology | Single-threaded Python workers       | Explore Go/Rust for performance  | Could handle high-load scraping faster     |
| Process    | End-to-end integration late          | Frequent integration demos       | Early validation of component interplay    |
| Scope      | Broader (scheduling + caching + all) | Focus on core scraping first     | Ensure core functionality is rock-solid    |

Personal Growth

Skills Developed

| Skill                  | Before Project | After Project |
|------------------------|----------------|---------------|
| API Design             | Intermediate   | Advanced      |
| Asynchronous Python    | Beginner       | Intermediate  |
| Docker & Deployment    | Beginner       | Intermediate  |
| LLM Prompt Engineering | Beginner       | Intermediate  |
| Automated Testing      | Beginner       | Advanced      |

Key Takeaways

- **Decompose large projects into microservices** to manage complexity and facilitate parallel development.
- **Automate testing and deployment** early to maintain quality and confidence in changes.
- **Clear communication between system parts** (via message queues and well-defined contracts) is critical for reliability.

Retrospective completed: 2025-01-05



# API Reference

---

## Base URL

---

- **Development:** <http://localhost:8000>
- All endpoints are under the /api/v1 path (for versioning) or at the root for authentication and health.

## Authentication

---

The API uses JWT-based auth. Obtain a token via /auth/token and include it as:

Authorization: Bearer <your\_access\_token>

## Endpoints

---

### Auth & Health

---

- **POST /auth/register**  
Create a new user account.  
**Body:** {"username": "user", "password": "pass123", "email": "[a@b.com](mailto:a@b.com)".  
**Success (200):** {"id": 1, "username": "user", "email": "a@b.com", "is\_active": true} .  
**Errors:** 400 if username exists; 429 if rate-limited.
- **POST /auth/token**  
Obtain a JWT token.  
**Form Data:** username=user&password=pass123 .  
**Success (200):** {"access\_token": "eyJ...", "token\_type": "bearer"} .  
**Errors:** 401 invalid credentials; 429 rate limit.
- **GET /**  
Root endpoint. Returns basic API info (service name, version).  
**Example Response (200):** {"message": "Intelligent Web Data Aggregator API", "version": "1.0.0", "status": "operational", ...} .
- **GET /health**  
Liveness probe. **Response:**  
{ "status": "healthy", "timestamp": "...", "service": "api", "version": "1.0.0", "uptime": "operational" } .
- **GET /api/v1/health**  
Readiness with DB check. **Response:** includes { "database": { "status": "connected", ... } } .

## Core API (Tasks)

---

(All below require Authorization: Bearer <token>)

- **POST /api/v1/process**

Create a new scraping task.

**Body:** {"url": "<target\_url>", "prompt": "<what to extract>"} .

**Rate-Limit:** 10/min per IP.

**Success (202):** {"task\_id": "<uuid>", "status": "PENDING", "message": "Task created successfully"} .

**Errors:** 400 if bad URL or prompt (including injection detected); 401 if not authenticated; 429 rate-limit.

- **GET /api/v1/status/{task\_id}**

Poll status of a task.

**Path Param:** task\_id (string, UUID).

**Success (200):** JSON with task info, e.g.:

```
{
  "task_id": "...",
  "status": "IN_PROGRESS",
  "progress": 40,
  "message": null,
  "created_at": "...",
  "updated_at": "..."
}
```

- Possible statuses: PENDING, IN\_PROGRESS, SUCCESS, FAILED.

**Errors:** 401 if not auth; 403 if not owner; 404 if unknown ID.

- **GET /api/v1/result/{task\_id}**

Retrieve results of a completed task.

**Path Param:** task\_id .

- If status=SUCCESS (200): Returns full data, e.g.:

```
{
  "task_id": "...",
  "status": "SUCCESS",
  "url": "https://example.com",
  "prompt": "...",
  "data": [
    { "text": "...", "source": "...", "confidence": 0.92 }
  ],
  "metadata": { "total_matches": 1, "used_cached_parser": false },
  "processing_time": 3.21,
  "created_at": "...", "completed_at": "..."
}
```

- If still processing (202): same structure as status endpoint with "status": "IN\_PROGRESS" .

**Errors:** 400 if task failed; 401/403 if unauthorized; 404 if not found.

## Scheduling

---

(All below require auth)

- **POST /api/v1/jobs**  
Create a scheduled (cron) scraping job.  
**Body:** {"url": "...", "prompt": "...", "schedule\_cron": "0 9 \* \* \*"} .  
**Success (200):** JSON object of the new job, e.g.:

```
{
  "id": 1,
  "url": "https://example.com",
  "prompt": "Get prices",
  "schedule_cron": "0 9 * * *",
  "next_run_at": "2025-08-09T09:00:00Z",
  "last_run_at": null,
  "created_at": "2025-08-08T12:00:00Z"
}
```

- (Next run is computed by the scheduler).
- **GET /api/v1/jobs**  
List all jobs for the current user.  
**Success (200):** JSON array of job objects (as above).
- **DELETE /api/v1/jobs/{job\_id}**  
Delete a scheduled job.  
**Path Param:** job\_id (integer).  
**Success (200):** {"message": "Job deleted"} .  
**Errors:** 404 if no such job or not owned by user.
- **GET /api/v1/users/me/activity**  
(Optional) Returns current user's recent tasks and jobs.  
**Success (200):** e.g.:

```
{
  "tasks": [{ "task_id": "...", "status": "SUCCESS", ... }],
  "scheduled_jobs": [{ "id":1, "url":"...", ...}]
}
```

- (Not in minimal API; included as a helpful endpoint if implemented).

Other Endpoints

- **POST /api/v1/test-task**  
(For internal/test use) Creates a dummy task to verify DB connectivity.  
**Response:** {"message": "Test task created successfully", "task": {...}}. Typically not used by end users.

Error Responses

The API uses standard HTTP error codes. Common errors include:

| Code | Description  |
|------|--------------|
| 200  | OK - success |

| Code | Description                                 |
|------|---------------------------------------------|
| 202  | Accepted - processing (async task started)  |
| 400  | Bad Request - validation failed             |
| 401  | Unauthorized - missing/invalid token        |
| 403  | Forbidden - no permission on this resource  |
| 404  | Not Found - invalid endpoint or ID          |
| 422  | Unprocessable - input validation (Pydantic) |
| 429  | Too Many Requests - rate limit exceeded     |
| 500  | Internal Server Error - unexpected failure  |

Rate limits are enforced per-IP: 10/min for /api/v1/process, 5/min for /auth/register, 10/min for /auth/token.

## Swagger/OpenAPI

---

Interactive API documentation is available at /docs (Swagger UI) once the service is running. The OpenAPI JSON is also exposed at /openapi.json for integration or client generation.

# Database Schema

## Overview

- **Database:** PostgreSQL 15 (managed via Docker container).
- **ORM:** SQLAlchemy (models defined in `shared/database/models.py`).
- **Schema Management:** Alembic is used for migrations (initial setup and subsequent changes).

## Entity Relationship Diagram

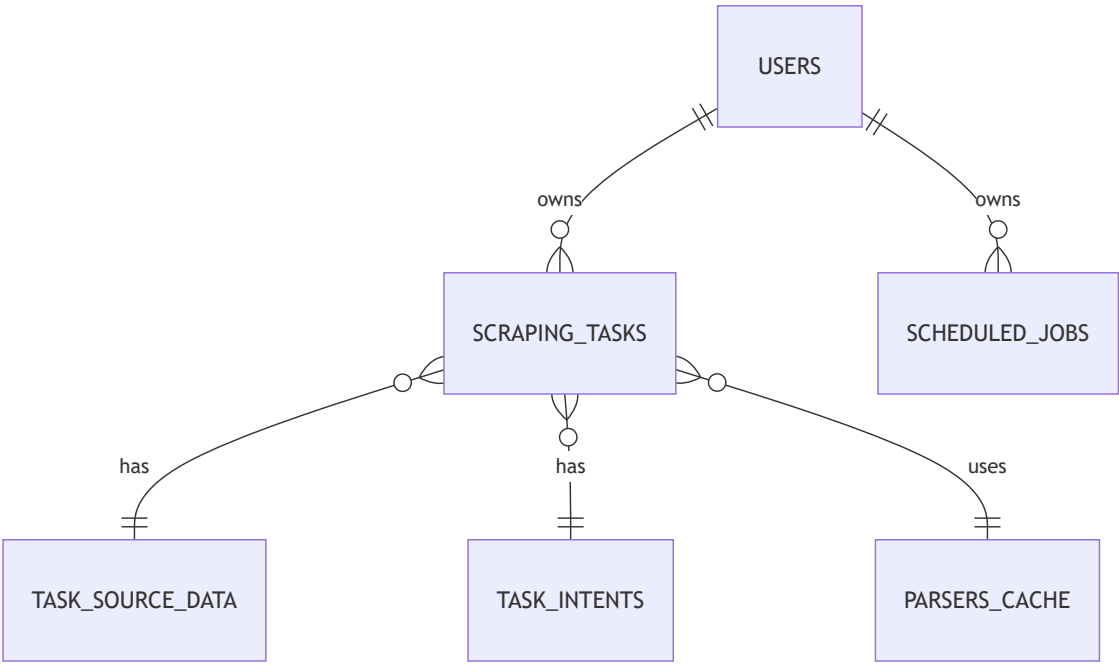


Figure: Simplified ER diagram showing core relationships (User-Tasks-Jobs, and task dependencies).

## Tables

### users

Stores user credentials and status.

| Column          | Type         | Constraints       | Description                |
|-----------------|--------------|-------------------|----------------------------|
| id              | INTEGER      | PK, AUTOINCREMENT | Unique user ID             |
| username        | VARCHAR(50)  | NOT NULL, UNIQUE  | Login name                 |
| hashed_password | VARCHAR(255) | NOT NULL          | Bcrypt hashed password     |
| email           | VARCHAR(255) | UNIQUE            | User's email (optional)    |
| is_active       | BOOLEAN      | DEFAULT TRUE      | Whether account is active  |
| created_at      | TIMESTAMP    | DEFAULT NOW()     | Account creation timestamp |

**Indexes:** on username, email.

## scraping\_tasks

Main table for tracking each scraping task.

| Column                                  | Type      | Constraints                      | Description                                         |
|-----------------------------------------|-----------|----------------------------------|-----------------------------------------------------|
| id                                      | INTEGER   | PK,<br>AUTOINCREMENT             | Primary key                                         |
| task_id                                 | VARCHAR   | UNIQUE, NOT<br>NULL              | External task UUID                                  |
| url                                     | TEXT      | NOT NULL                         | Target page URL                                     |
| user_prompt                             | TEXT      | NOT NULL                         | Original user prompt                                |
| status                                  | VARCHAR   | NOT NULL<br>DEFAULT<br>'PENDING' | Task status<br>(PENDING/IN_PROGRESS/SUCCESS/FAILED) |
| error_message                           | TEXT      | NULLABLE                         | Error text if failed                                |
| extracted_data                          | JSON      | NULLABLE                         | Final data array (on SUCCESS)                       |
| total_matches                           | INTEGER   | NULLABLE                         | Number of items extracted                           |
| processing_time_seconds                 | INTEGER   | NULLABLE                         | Time taken (seconds)                                |
| used_cached_parser                      | BOOLEAN   | NOT NULL<br>DEFAULT FALSE        | Whether a cached selector was used                  |
| created_at, started_at,<br>completed_at | TIMESTAMP |                                  | Timestamps for creation, start, completion          |
| owner_id                                | INTEGER   | FK -> users.id                   | Ownering user (nullable if anonymous tasks)         |
| intent_id                               | INTEGER   | FK -><br>task_intents.id         | Intent details (for query reuse)                    |
| used_parser_id                          | INTEGER   | FK -><br>parsers_cache.id        | Cached parser used (if any)                         |

**Relationships:** Each task optionally links to one TaskIntent (normalized prompt data) and one ParserCache entry (if cached selector was used).

**task\_intents**

Normalizes the components of user prompts. Each task may reference one intent.

| Column          | Type    | Constraints       | Description                          |
|-----------------|---------|-------------------|--------------------------------------|
| id              | INTEGER | PK, AUTOINCREMENT | PK                                   |
| target          | TEXT    | NULLABLE          | Main object (e.g. "job listings")    |
| keywords        | JSON    | NULLABLE          | Array of keywords                    |
| schema_fields   | JSON    | NULLABLE          | Array of requested field names       |
| constraints     | JSON    | NULLABLE          | Additional conditions (e.g. filters) |
| output_shape    | TEXT    | NULLABLE          | Human-readable description of output |
| normalized_hash | VARCHAR | NULLABLE INDEX    | Hash for intent matching             |

**task\_source\_data**

Holds large source data for a task to keep scraping\_tasks slim.

| Column            | Type    | Constraints                   | Description                            |
|-------------------|---------|-------------------------------|----------------------------------------|
| id                | INTEGER | PK                            | PK                                     |
| task_id           | INTEGER | FK -> scraping_tasks.id (1:1) | Associated scraping task               |
| page_content      | TEXT    | NULLABLE                      | innerText of page (structured text)    |
| html_content      | TEXT    | NULLABLE                      | Full HTML of the page                  |
| network_requests  | JSON    | NULLABLE                      | Captured XHR/fetch responses (as JSON) |
| chosen_source_url | TEXT    | NULLABLE                      | URL of content used (if multiple)      |

**parsers\_cache**

Caches successful selectors to reuse.

| Column          | Type    | Constraints      | Description                   |
|-----------------|---------|------------------|-------------------------------|
| id              | INTEGER | PK               | PK                            |
| url_pattern     | VARCHAR | NOT NULL         | Pattern (domain or URL regex) |
| domain_id       | INTEGER | FK -> domains.id | Foreign key to domains table  |
| user_intent     | TEXT    | NOT NULL         | Normalized prompt text        |
| intent_keywords | JSON    | NULLABLE         | Keywords for matching         |

| Column                      | Type      | Constraints              | Description                                                         |
|-----------------------------|-----------|--------------------------|---------------------------------------------------------------------|
| target_data_type            | VARCHAR   | NULLABLE                 | e.g. "job_listings"                                                 |
| generated_selector          | TEXT      | NOT NULL                 | The extraction pattern (CSS selector, Regex, or JSON path)          |
| source_type                 | VARCHAR   | NOT NULL                 | Type of content the selector applies to (e.g. SEMANTIC, HTML, JSON) |
| source_identifier           | TEXT      | NULLABLE                 | e.g. XHR URL or HTML context                                        |
| test_matches_count          | INTEGER   | NOT NULL                 | # of matches found during test                                      |
| confidence_score            | INTEGER   | NOT NULL<br>DEFAULT 100  | Match percentage (0-100)                                            |
| times_used                  | INTEGER   | NOT NULL<br>DEFAULT 0    | Usage count                                                         |
| is_active                   | BOOLEAN   | NOT NULL<br>DEFAULT TRUE | If parser is still valid                                            |
| created_at,<br>last_used_at | TIMESTAMP | DEFAULT NOW()            | Timestamps                                                          |

Each cached parser links one-to-many to scraping\_tasks that used it.

## scheduled\_jobs

Stores user-defined cron tasks.

| Column        | Type      | Constraints               | Description                 |
|---------------|-----------|---------------------------|-----------------------------|
| id            | INTEGER   | PK                        | PK                          |
| url           | TEXT      | NOT NULL                  | Target page URL             |
| prompt        | TEXT      | NOT NULL                  | User's extraction prompt    |
| schedule_cron | VARCHAR   | NOT NULL                  | Cron expression             |
| is_active     | BOOLEAN   | NOT NULL DEFAULT TRUE     | If the job is active        |
| last_run_at   | TIMESTAMP | NULLABLE                  | Timestamp of last execution |
| next_run_at   | TIMESTAMP | NULLABLE                  | Next scheduled run time     |
| owner_id      | INTEGER   | FK -> users.id (NOT NULL) | Owning user                 |
| created_at    | TIMESTAMP | DEFAULT NOW()             | Job creation time           |

## Relationships

- **User → ScrapingTasks:** One-to-many (a user can have multiple tasks).



- **User** → **ScheduledJobs**: One-to-many (a user's jobs).
- **ScrapingTask** → **TaskIntent**: Many tasks can share one intent (the normalized prompt).
- **ScrapingTask** → **ParserCache**: (Optional) Many tasks can reuse the same parser entry.
- **ScheduledJobs** → **(ScrapingTask)**: Jobs spawn new tasks on schedule (history in separate log table, not shown).

## Migrations

---

Schema migrations are managed by Alembic. The first migration created all tables above. Subsequent migrations (if any) update columns or add indexes.

## Seeding

---

No default seed data is required. A test user and tasks can be added via scripts or directly in SQL for testing.

# Glossary

| Term             | Definition                                                                                                                                                             |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API              | Application Programming Interface - The set of HTTP endpoints provided by the backend for communication between the client and server.                                 |
| LLM              | Large Language Model - An AI model (like GPT-4 or Gemini) that understands and generates text; used here to interpret user prompts and generate data extraction logic. |
| Regex            | Regular Expression - A sequence of characters defining a search pattern. Used to extract structured data from HTML content.                                            |
| Semantic Content | Structured page text with role markers (e.g. [LINK:], ## headings) produced by the headless browser for cleaner LLM input.                                             |
| Celery           | An asynchronous task queue/job queue based on distributed message passing. Used to process scraping and AI tasks.                                                      |
| Redis            | In-memory data store used as a message broker for Celery and as a cache for rate-limiting.                                                                             |
| JWT              | JSON Web Token - A compact, URL-safe means of representing claims to be transferred between two parties. Used here for stateless API authentication.                   |
| Pydantic         | A Python library for data validation using type annotations. FastAPI uses it to define request/response schemas (models).                                              |
| Swagger UI       | Automatically generated web interface for interacting with the API; provided by FastAPI at /docs.                                                                      |
| Rate Limiting    | Technique to limit the number of requests a user or IP can make in a time period, to prevent abuse. Implemented via slowapi in the API.                                |
| Playwright       | A headless browser automation library used to fetch and render web pages (even with JavaScript).                                                                       |
| Docker Compose   | A tool for defining and running multi-container Docker applications. Used to orchestrate services (API, DB, Redis, etc.).                                              |
| Prompt Injection | A form of attack where malicious input attempts to alter the instructions given to the LLM. The system sanitizes prompts to prevent this.                              |
| Parser Cache     | A store of previously generated selectors (parsers) for reuse on repeated tasks, improving performance.                                                                |

# Acronyms

| Acronym     | Full Form                         | Description                                                                         |
|-------------|-----------------------------------|-------------------------------------------------------------------------------------|
| <b>API</b>  | Application Programming Interface | A set of endpoints for programmatic interaction with the backend.                   |
| <b>UI</b>   | User Interface                    | The part of the system the user interacts with (none provided in this project).     |
| <b>CSV</b>  | Comma-Separated Values            | A common text format for tabular data (can be generated from JSON results).         |
| <b>JSON</b> | JavaScript Object Notation        | A lightweight data-interchange format. Used for all API requests/responses.         |
| <b>ORM</b>  | Object-Relational Mapping         | Technique for mapping database tables to code classes (SQLAlchemy is the ORM here). |

# Domain-Specific Terms

---

## Web Scraping

---

- **Scraping Task:** A user-request to extract data from a URL based on a prompt. Represented by a ScrapingTask record.
- **Extraction Pattern:** The regex or selectors generated by the LLM to find the requested data on the page.

## Data Extraction

---

- **Single-field Extraction:** The pipeline used when the prompt asks for one field; involves finding example text and generating a focused regex.
- **Schema Extraction:** The pipeline for multi-field requests; LLM returns structured output for all fields at once with relationships preserved.

*Document created: 2025-01-05*